CAR RENTAL RESERVATION AND VEHICLE RETURN MANAGEMENT SYSTEM

Design and Implementation of a Database Management System

A DBMS Project



Submitted by

Punit Vaibhava Ram Reddy – 25WU0104018

V Charan Karthikeya – 25WU0102296

A Karthikeya Varma - 25WU0103001

Y. Charan Kumar Reddy – 25WU0102313

Department of Computer Science and Engineering

Course: Database Management Systems

Faculty: Dr. Kiran Mayee

Academic Year: 2026–2027

Table of Contents

Abstract **3**
I. Introduction **4**
II. Problem Statement **5**
III. Objectives and Scope **6**
A. Objectives 6
B. Scope 6
IV. System Requirements **7**
A. Software Requirements 7
B. Hardware Requirements 7
C. Functional and Non-Functional Requirements 7
V. System Design **9**
A. Entity-Relationship Diagram 9
B. Relational Schema 11
C. Normalization 11
D. Data Dictionary 12
E. Schema Relationships and Data Integrity 14
VI. Implementation **15**
A. Technology Stack 15
B. SQL Commands 15
C. Database Connectivity 22
D. Triggers, Transactions and Concurrency 22
E. Validation and Error Handling 24
F. System Architecture and Data Flow 24
G. Security, Indexing and Backup 25
VII. Testing **27**
VIII. Conclusion and Future Work **29**
Contribution of Team Members **30**
Appendix A: Source Code Repository **31**
References **31**

Abstract

Car rental is one of the fastest-growing segments of urban mobility, serving travellers, tourists and businesses every day. Rental companies, however, still depend on paper registers and scattered spreadsheets to manage reservations, fleet availability, vehicle pickups, returns and billing. These manual methods lead to double booking of vehicles, delayed return processing, inconsistent customer records and billing disputes. This project, the **Car Rental Reservation and Vehicle Return Management System**, addresses these problems by designing and implementing a centralized relational database that supports the complete rental lifecycle: searching for vehicles by branch, category and dates, reserving and cancelling bookings, issuing a rental agreement at pickup, recording the vehicle return with odometer, fuel and damage inspection, calculating late-return fees, and recording payments and deposits.

The database is organised around seven entities: Customer, Vehicle, Branch, Reservation, Rental Agreement, Vehicle Return and Payment. It was modelled with an Entity-Relationship diagram, converted to a relational schema and normalized up to Third Normal Form (3NF). Data integrity is enforced through primary keys, foreign keys, UNIQUE, NOT NULL and CHECK constraints, while triggers prevent overlapping reservations for the same vehicle, apply late fees automatically and keep the vehicle and reservation status synchronized. Critical operations such as booking and return are executed as ACID transactions with rollback on failure. Sample data and SQL queries for availability search, return summaries, revenue per branch and billing were executed and verified, and a set of test cases confirmed that invalid operations are rejected. The project shows how a carefully designed database improves reliability, speed and transparency in car rental operations.

Keywords: Car rental, DBMS, ER diagram, normalization, SQL, reservation, vehicle return, triggers, transactions, data integrity.

I. Introduction

Mobility services have changed the way people travel inside and between cities. Car rental companies now operate fleets of hundreds of vehicles across several branches and serve customers who expect to search, book and return a car within minutes. Behind every successful booking there is a large amount of data that must be accurate at all times: which customer holds which licence, which vehicle is parked at which branch, which dates are already reserved, what the odometer and fuel level were at pickup, whether the car came back damaged, and how much the customer owes. A well-designed database is the backbone of any modern rental system.

This project develops a simplified but realistic database for a **Car Rental Reservation and Vehicle Return Management System**. The system allows branch staff and customers to search available cars, create and cancel reservations, convert a reservation into a rental agreement at pickup, process the return of the vehicle with an inspection, and calculate the final bill including late-return charges. The focus of the work is on correct database design, data integrity, prevention of double booking, consistent transaction handling and meaningful SQL reporting.

A. Operational Areas Covered

The system supports three closely connected areas of a rental business, summarized in Table I. Each area maps directly to one or more tables of the database designed later in this report.

TABLE I. OPERATIONAL AREAS SUPPORTED BY THE DATABASE

Area	What the database supports	Main tables involved
Reservation	Search cars by branch, category and dates; check real-time availability; book, modify and cancel reservations; track status (Pending / Confirmed / Cancelled / Completed).	CUSTOMER, VEHICLE, BRANCH, RESERVATION
Vehicle Return	Record pickup and return time; odometer and fuel-level checks; damage inspection notes; late-return fee calculation.	RENTAL, RETURN_RECORD
Data Management	Customer and licence records; fleet and branch details; payments and deposits; fast reporting with SQL.	CUSTOMER, VEHICLE, BRANCH, PAYMENT

B. Organization of the Report

The report is organized as follows. Section II states the problem and the proposed solution. Section III lists the objectives, scope and user roles. Section IV gives the software, hardware and functional requirements. Section V presents the database design: the ER diagram, relational schema, normalization, data dictionary and integrity constraints. Section VI explains the implementation, including DDL, sample data, SQL queries, triggers, transactions, system architecture and security. Section VII reports the testing carried out, and Section VIII concludes the report and lists the future work.

II. Problem Statement

Rental businesses that rely on manual registers, paper agreements and loosely connected spreadsheets face a number of operational problems:

- **Double booking:** the same vehicle can be promised to two customers for overlapping dates because there is no single, reliable view of reservations.
- **Manual registers and paper agreements:** records are slow to search, easy to lose and hard to correct.
- **Scattered customer and rental records:** the same customer appears several times with different spellings of name, phone and licence number.
- **Delayed return processing and billing:** staff calculate rental charges and late fees by hand, which is slow and error prone.
- **No real-time view of fleet availability:** branches cannot see which cars are free, rented or under maintenance.
- **Poor tracking of damage and fuel at return:** without a recorded odometer and condition at pickup and return, damage and fuel disputes cannot be resolved.
- **Data inconsistency and duplicate records:** the absence of constraints allows invalid dates, negative rates and orphan records.
- **Difficult reporting and analytics:** questions such as revenue per branch or utilization of the fleet require hours of manual consolidation.

There is therefore a need for a centralized, normalized and secure relational database that automates reservation and return management while guaranteeing accuracy, consistency and quick retrieval of information. Table II maps every challenge to the corresponding feature of the proposed solution.

TABLE II. CURRENT CHALLENGES AND THE PROPOSED SOLUTION

Current challenge	Proposed solution
Double booking of the same vehicle	No-overlap rule on (vehicle_id, start_date, end_date) enforced by a trigger and row locking
Manual registers and paper agreements	Centralized relational database as a single source of truth
Scattered customer and rental records	One CUSTOMER record per person, linked by foreign keys to reservations
Delayed return processing and billing	Automated rental charge and late-fee billing; instant return check-in
No real-time view of fleet availability	Availability query and vehicle status (AVAILABLE / RENTED / MAINTENANCE)
Poor tracking of damage and fuel at return	RENTAL and RETURN_RECORD tables log odometer, fuel level and damage notes
Data inconsistency and duplicate records	Primary, foreign, UNIQUE, NOT NULL and CHECK constraints; 3NF design
Difficult to generate reports and analytics	SQL joins and aggregate queries for utilization and revenue reports

III. Objectives and Scope

A. Objectives

The proposed system is a structured database that manages car rental reservations and vehicle returns end to end. Its main objectives are:

- Design a normalized relational database (up to 3NF) for car rental reservation and vehicle return management.
- Allow customers and staff to search available cars by branch, category and dates.
- Reserve, modify and cancel bookings while preventing double booking of vehicles.
- Issue a rental agreement at pickup and record odometer, fuel level and deposit.
- Process the vehicle return with an inspection (odometer, damage notes) and return branch.
- Calculate rental charges, deposit and late fees automatically.
- Maintain customer information, including contact details and unique driving-licence numbers.
- Support concurrent bookings using transactions, with backup and recovery.
- Provide SQL queries for availability, return summaries, utilization and revenue reports.

The role of the database in achieving these objectives is summarized below: it provides centralized data storage, enforces data integrity, supports efficient queries and joins, maintains relationships between entities, supports concurrent bookings through transactions, and allows backup and recovery.

B. Scope

In scope: customer and licence records, branch and fleet management, vehicle availability search, reservation creation and cancellation, rental agreement at pickup, vehicle return and inspection, late-fee calculation, payment and deposit records, status tracking, integrity constraints, triggers, transactions and SQL reports.

Out of scope (future work): real payment-gateway integration, live GPS tracking of vehicles, dynamic pricing, a finished mobile application and integration with external travel portals. Payments are stored as records of a transaction (amount, mode and status) and are not processed by a real gateway.

C. Users and Their Roles

Three types of users interact with the system. Table III lists what each of them can do; role-based access control (Section VI.G) restricts each role to these operations.

TABLE III. USER ROLES AND RESPONSIBILITIES

Role	Responsibilities	Tables accessed
Customer	Search cars, reserve and cancel own bookings, view own rental and payment history	VEHICLE (read), RESERVATION, PAYMENT (own rows)
Branch Staff	Confirm reservations, issue rental agreements, process returns and inspections, record payments	RESERVATION, RENTAL, RETURN_RECORD, PAYMENT, VEHICLE
Administrator	Manage branches, fleet and rates, view reports, manage users and backups	All tables

IV. System Requirements

A. Software Requirements

TABLE IV. SOFTWARE REQUIREMENTS

Requirement	Technology
Operating System	Windows 10/11 (Linux also supported)
Database (RDBMS)	MySQL Server 8.0 (ANSI SQL, InnoDB engine for transactions and foreign keys)
Query / Admin Tool	MySQL Workbench or MySQL command-line client
Application layer (proposed)	Python with Flask and mysql-connector-python
User interface (proposed)	Web / mobile front end using HTML, CSS and JavaScript
Diagrams and documentation	Draw.io / Matplotlib for diagrams, MS Word for the report
Version Control	Git and GitHub

The database layer described in this report is complete and tested. The application and user-interface layers are listed as proposed technologies and form the next phase of the project (Section VIII).

B. Hardware Requirements

TABLE V. HARDWARE REQUIREMENTS

Requirement	Minimum
Processor	Intel i5 or equivalent
RAM	4 GB or above
Storage	10 GB available
Internet	Required for GitHub and package installation

C. Functional and Non-Functional Requirements

The functional requirements derived from the problem statement are listed in Table VI and the quality requirements in Table VII.

TABLE VI. FUNCTIONAL REQUIREMENTS

ID	Requirement	Supported by
FR1	Register and store customer details with a unique phone and licence number	CUSTOMER
FR2	Maintain branches and the fleet (registration, model, category, daily rate, status)	BRANCH, VEHICLE
FR3	Search available vehicles at a branch for a date range	Availability query (Q1)
FR4	Create, modify and cancel reservations; reject overlapping bookings	RESERVATION + trigger
FR5	Issue a rental agreement at pickup with odometer, fuel level and deposit	RENTAL
FR6	Process the return with odometer, damage notes and return branch	RETURN_RECORD
FR7	Calculate late fee and final bill automatically	Trigger, billing query (Q7)
FR8	Record payments, mode and status	PAYMENT
FR9	Generate reports: revenue per branch, utilization, pending reservations	Queries Q3 to Q6

TABLE VII. NON-FUNCTIONAL REQUIREMENTS

Attribute	Requirement
Integrity	No orphan records, no duplicate vehicles or licences, no invalid dates or rates
Consistency	Booking, vehicle blocking and deposit are committed together or not at all (ACID)
Performance	Indexes on vehicle, branch and date columns keep availability search fast
Security	Role-based access; encrypted payment and licence data
Reliability	Regular backups and a tested recovery procedure
Scalability	Normalized schema that can be extended with new branches, categories and services

V. System Design

A. Entity-Relationship Diagram

The conceptual design identifies **seven core entities**. Table VIII describes each entity, and Fig. 1 shows the complete ER diagram with attributes, primary keys (PK), foreign keys (FK) and the cardinality of every relationship.

TABLE VIII. ENTITIES OF THE SYSTEM

Entity	Description
CUSTOMER	Renter details: name, contact information and unique driving-licence number
VEHICLE	A car in the fleet: registration number, make and model, category, daily rate, home branch and current status
BRANCH	A rental location with city, state and operating status; also the pickup and return point
RESERVATION	A booking made by a customer for a vehicle and a date range, with a status
RENTAL	The rental agreement created at actual pickup: pickup time, odometer, fuel level and security deposit
RETURN_RECORD	The vehicle return: return time and branch, closing odometer, damage notes and late fee
PAYMENT	A payment transaction for a rental: amount, mode (UPI, card, cash) and status

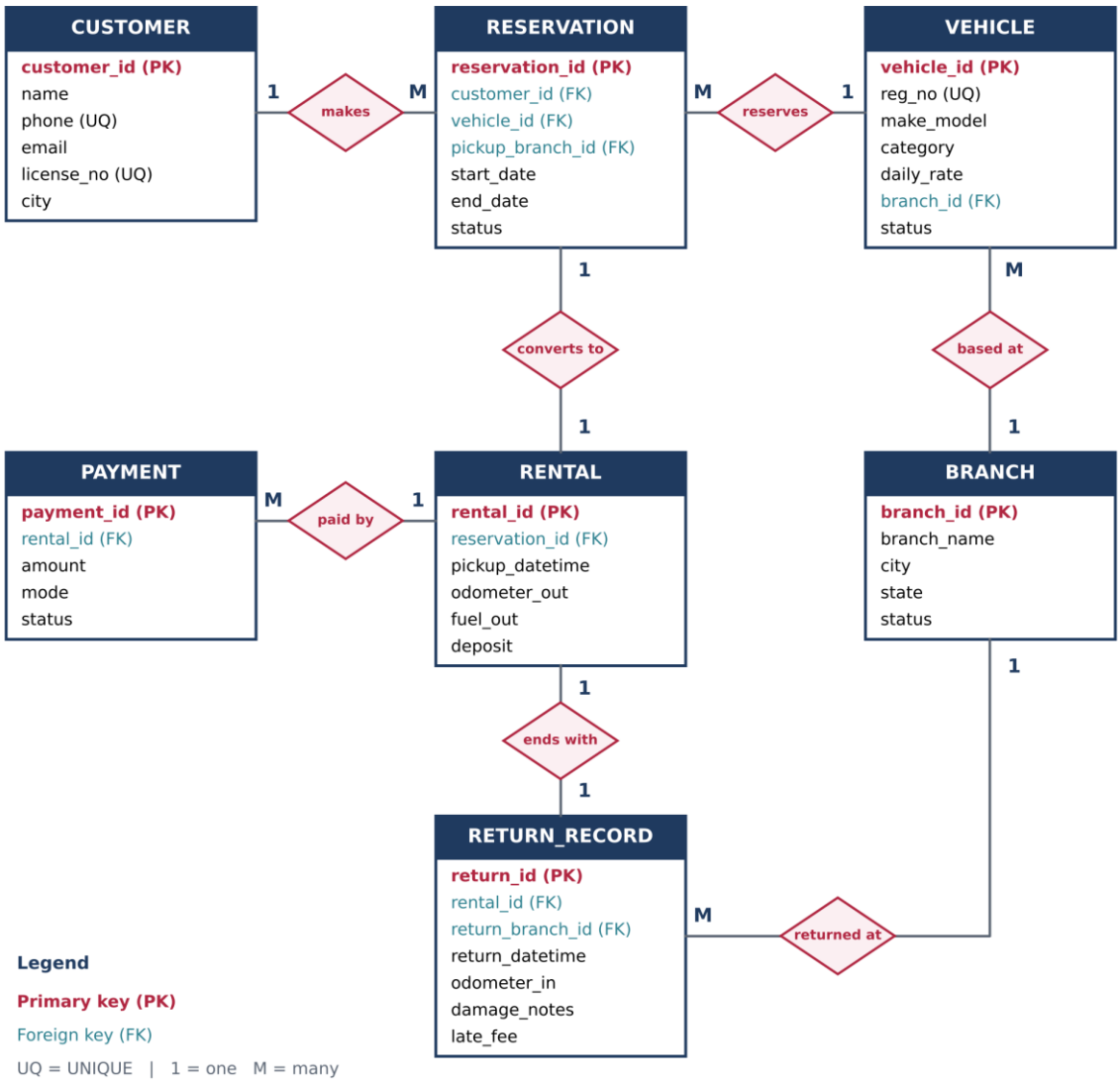


Fig. 1. Entity-Relationship diagram of the Car Rental Reservation and Vehicle Return Management System.

The ER diagram is read as follows. Primary keys uniquely identify records, while foreign keys implement the relationships between entities. Table IX lists every relationship with its cardinality and the foreign key that implements it.

TABLE IX. RELATIONSHIPS AND CARDINALITIES

Entities	Relationship	Cardinality	Implemented by
CUSTOMER – RESERVATION	makes	1 : M	RESERVATION.customer_id (FK)
VEHICLE – RESERVATION	reserves	1 : M	RESERVATION.vehicle_id (FK)
BRANCH – VEHICLE	based at	1 : M	VEHICLE.branch_id (FK)
BRANCH – RESERVATION	pickup at	1 : M	RESERVATION.pickup_branch_id (FK)
RESERVATION – RENTAL	converts to	1 : 1	RENTAL.reservation_id (FK, UNIQUE)
RENTAL – RETURN_RECORD	ends with	1 : 1	RETURN_RECORD.rental_id (FK, UNIQUE)

Entities	Relationship	Cardinality	Implemented by
RENTAL – PAYMENT	paid by	1 : M	PAYMENT.rental_id (FK)
BRANCH – RETURN_RECORD	returned at	1 : M	RETURN_RECORD.return_branch_id (FK)

Key observations: (i) one customer can make many reservations, but each reservation belongs to exactly one customer; (ii) a vehicle can appear in many reservations over time, but never in two overlapping ones; (iii) a reservation produces at most one rental, and a rental ends with at most one return; (iv) a rental can be settled by more than one payment (for example a deposit and a final bill).

B. Relational Schema

The ER model was converted into the relations shown in Table X. Primary keys are underlined in the SQL by the PRIMARY KEY clause and foreign keys reference the parent table. The table that records a vehicle return is named `RETURN_RECORD` because `RETURN` is a reserved word in SQL.

TABLE X. RELATIONAL SCHEMA

Relation	Attributes	Keys
CUSTOMER	customer_id, name, phone, email, license_no, city	PK: customer_id UNIQUE: phone, license_no
BRANCH	branch_id, branch_name, city, state, status	PK: branch_id
VEHICLE	vehicle_id, reg_no, make_model, category, daily_rate, branch_id, status	PK: vehicle_id UNIQUE: reg_no FK: branch_id
RESERVATION	reservation_id, customer_id, vehicle_id, pickup_branch_id, start_date, end_date, status	PK: reservation_id FK: customer_id, vehicle_id, pickup_branch_id
RENTAL	rental_id, reservation_id, pickup_datetime, odometer_out, fuel_out, deposit	PK: rental_id FK: reservation_id (UNIQUE)
RETURN_RECORD	return_id, rental_id, return_branch_id, return_datetime, odometer_in, damage_notes, late_fee	PK: return_id FK: rental_id (UNIQUE), return_branch_id
PAYMENT	payment_id, rental_id, amount, mode, status	PK: payment_id FK: rental_id

Note: identifiers are stored as short codes (for example c01, v101, br01, r701) so that records are easy to read in reports; they are declared as VARCHAR(10) in the implementation.

C. Normalization

The database was designed using normalization principles to reduce redundancy and improve consistency. Consider a single flat table `RENTAL_ALL` that stores everything about a rental in one row. Table XI shows why such a design is unsafe: the customer details and the branch details are repeated for every rental.

TABLE XI. UNNORMALIZED FLAT TABLE (ILLUSTRATION OF THE PROBLEM)

Rental	Customer	Phone	Vehicle	Branch name	Branch city	Dates
rt901	Rahul Verma	9876543210	Hyundai Creta	Hyderabad Airport Hub	Hyderabad	18–21 Oct
rt905	Rahul Verma	9876543210	Toyota Innova	Hyderabad Airport Hub	Hyderabad	02–04 Nov
rt906	Rahul Verma	9876543219	Hyundai Creta	Hyderabad Airport Hub	Hyderabad	10–12 Nov

Illustrative rows only. Rahul's phone number appears three times and is already inconsistent in the third row (update anomaly); a new branch cannot be stored until a rental exists (insertion anomaly); and deleting the only rental of a branch deletes the branch information (deletion anomaly). Normalization removes these anomalies:

- **First Normal Form (1NF):** every attribute holds a single atomic value (for example phone, city and licence number are separate columns) and there are no repeating groups. Each table has a primary key.
- **Second Normal Form (2NF):** all non-key attributes depend on the *whole* primary key. Customer data is moved to CUSTOMER, vehicle data to VEHICLE and branch data to BRANCH, so no attribute depends on only part of a key.
- **Third Normal Form (3NF):** non-key attributes depend only on the primary key, not on other non-key attributes. For example, branch city and state are not stored in VEHICLE; they are reached through `branch_id`, which removes the transitive dependency vehicle → branch → city.

As a result, **branch data is stored once and reused everywhere**, redundancy is eliminated, update, insert and delete anomalies are avoided and consistency is maintained. All seven relations of Table X are in 3NF.

D. Data Dictionary

The data dictionary describes every column of every table with its data type, constraint and meaning. Table XII to Table XVIII cover the seven tables.

TABLE XII. CUSTOMER TABLE

Column	Data Type	Constraint	Description
`customer_id`	VARCHAR(10)	PRIMARY KEY	Unique customer identifier
`name`	VARCHAR(50)	NOT NULL	Full name of the renter
`phone`	VARCHAR(15)	NOT NULL, UNIQUE	Contact number
`email`	VARCHAR(60)	—	E-mail address (optional)
`license_no`	VARCHAR(20)	NOT NULL, UNIQUE	Driving-licence number
`city`	VARCHAR(40)	—	City of residence

TABLE XIII. BRANCH TABLE

Column	Data Type	Constraint	Description
`branch_id`	VARCHAR(10)	PRIMARY KEY	Unique branch identifier
`branch_name`	VARCHAR(60)	NOT NULL	Name of the rental location
`city`	VARCHAR(40)	NOT NULL	City
`state`	VARCHAR(40)	NOT NULL	State
`status`	VARCHAR(10)	NOT NULL, DEFAULT ACTIVE	ACTIVE or INACTIVE

TABLE XIV. VEHICLE TABLE

Column	Data Type	Constraint	Description
`vehicle_id`	VARCHAR(10)	PRIMARY KEY	Unique vehicle identifier
`reg_no`	VARCHAR(15)	NOT NULL, UNIQUE	Registration number plate
`make_model`	VARCHAR(40)	NOT NULL	Make and model
`category`	VARCHAR(20)	NOT NULL	Hatchback, SUV, MPV, …

Column	Data Type	Constraint	Description
daily_rate	DECIMAL(8,2)	NOT NULL, CHECK (> 0)	Rental price per day
branch_id	VARCHAR(10)	FOREIGN KEY	Home branch of the vehicle
status	VARCHAR(15)	CHECK IN (AVAILABLE, RENTED, MAINTENANCE)	Current availability

TABLE XV. RESERVATION TABLE

Column	Data Type	Constraint	Description
reservation_id	VARCHAR(10)	PRIMARY KEY	Unique booking identifier
customer_id	VARCHAR(10)	FOREIGN KEY	Customer who booked
vehicle_id	VARCHAR(10)	FOREIGN KEY	Vehicle booked
pickup_branch_id	VARCHAR(10)	FOREIGN KEY	Branch where the car will be picked up
start_date	DATE	NOT NULL	First day of the booking
end_date	DATE	NOT NULL, CHECK (>= start_date)	Last day of the booking
status	VARCHAR(15)	CHECK IN (PENDING, CONFIRMED, CANCELLED, COMPLETED)	Booking status

TABLE XVI. RENTAL TABLE

Column	Data Type	Constraint	Description
rental_id	VARCHAR(10)	PRIMARY KEY	Unique rental agreement identifier
reservation_id	VARCHAR(10)	FOREIGN KEY, UNIQUE	Reservation converted into this rental
pickup_datetime	DATETIME	NOT NULL	Actual pickup date and time
odometer_out	INT	NOT NULL	Odometer reading at pickup (km)
fuel_out	VARCHAR(10)	—	Fuel level at pickup (e.g. Full)
deposit	DECIMAL(8,2)	NOT NULL, CHECK (>= 0)	Refundable security deposit

TABLE XVII. RETURN_RECORD TABLE

Column	Data Type	Constraint	Description
return_id	VARCHAR(10)	PRIMARY KEY	Unique return identifier
rental_id	VARCHAR(10)	FOREIGN KEY, UNIQUE	Rental being closed
return_branch_id	VARCHAR(10)	FOREIGN KEY	Branch where the car was returned
return_datetime	DATETIME	NOT NULL	Actual return date and time
odometer_in	INT	NOT NULL	Odometer reading at return (km)
damage_notes	VARCHAR(100)	—	Inspection notes (None if no damage)
late_fee	DECIMAL(8,2)	DEFAULT 0, CHECK (>= 0)	Late-return charge (set by trigger)

TABLE XVIII. PAYMENT TABLE

Column	Data Type	Constraint	Description
payment_id	VARCHAR(10)	PRIMARY KEY	Unique payment identifier
rental_id	VARCHAR(10)	FOREIGN KEY	Rental being paid for
amount	DECIMAL(8,2)	NOT NULL, CHECK (> 0)	Amount paid
mode	VARCHAR(20)	NOT NULL	UPI, CARD, CASH, …
status	VARCHAR(15)	NOT NULL	SUCCESS, FAILED, PENDING

E. Schema Relationships and Data Integrity

Referential links and integrity rules guarantee that the stored data always remains valid. The foreign-key relationships are: CUSTOMER.customer_id → RESERVATION.customer_id; VEHICLE.vehicle_id → RESERVATION.vehicle_id; BRANCH.branch_id → VEHICLE.branch_id, RESERVATION.pickup_branch_id and RETURN_RECORD.return_branch_id; RESERVATION.reservation_id → RENTAL.reservation_id; and RENTAL.rental_id → RETURN_RECORD.rental_id and PAYMENT.rental_id. Table XIX summarizes the integrity constraints used.

TABLE XIX. DATA INTEGRITY CONSTRAINTS

Type	Rule	Example
Primary key	Every row is uniquely identified	customer_id, vehicle_id, reservation_id
Referential (FK)	A child row must refer to an existing parent row; a referenced parent cannot be deleted	RESERVATION.customer_id must exist in CUSTOMER
Unique	No duplicates in identifying attributes	reg_no, license_no, phone, one rental per reservation
Domain (CHECK)	Values must be valid	daily_rate > 0; end_date >= start_date; status in allowed list
NOT NULL	Mandatory fields must be supplied	name, phone, start_date, pickup_datetime
Business rule (trigger)	No two active reservations for the same vehicle on overlapping dates	trg_no_overlap (Section VI.D)

VI. Implementation

A. Technology Stack

The database is implemented in **MySQL 8.0** using the InnoDB storage engine, which provides transactions, row-level locking and foreign-key enforcement. SQL statements are written in standard SQL with MySQL syntax for triggers. A Python backend with `mysql-connector-python` and a web front end are the proposed application layers (Section VI.C and VIII). All design diagrams in this report were produced from the final schema.

B. SQL Commands

1) DDL Commands

Data Definition Language commands create the database, the seven tables with their keys and constraints, and the supporting indexes.

```
CREATE DATABASE car_rental_db;
USE car_rental_db;

CREATE TABLE CUSTOMER (
  customer_id VARCHAR(10) PRIMARY KEY,
  name        VARCHAR(50) NOT NULL,
  phone       VARCHAR(15) NOT NULL UNIQUE,
  email       VARCHAR(60),
  license_no  VARCHAR(20) NOT NULL UNIQUE,
  city        VARCHAR(40)
);

CREATE TABLE BRANCH (
  branch_id   VARCHAR(10) PRIMARY KEY,
  branch_name VARCHAR(60) NOT NULL,
  city        VARCHAR(40) NOT NULL,
  state       VARCHAR(40) NOT NULL,
  status      VARCHAR(10) NOT NULL DEFAULT 'ACTIVE'
);

CREATE TABLE VEHICLE (
  vehicle_id  VARCHAR(10) PRIMARY KEY,
  reg_no      VARCHAR(15) NOT NULL UNIQUE,
  make_model  VARCHAR(40) NOT NULL,
  category    VARCHAR(20) NOT NULL,
  daily_rate  DECIMAL(8,2) NOT NULL CHECK (daily_rate > 0),
  branch_id   VARCHAR(10) NOT NULL,
  status      VARCHAR(15) NOT NULL DEFAULT 'AVAILABLE'
              CHECK (status IN ('AVAILABLE','RENTED','MAINTENANCE')),
  FOREIGN KEY (branch_id) REFERENCES BRANCH(branch_id)
);

CREATE TABLE RESERVATION (
  reservation_id   VARCHAR(10) PRIMARY KEY,
  customer_id      VARCHAR(10) NOT NULL,
  vehicle_id       VARCHAR(10) NOT NULL,
  pickup_branch_id VARCHAR(10) NOT NULL,
  start_date       DATE NOT NULL,
  end_date         DATE NOT NULL,
  status           VARCHAR(15) NOT NULL DEFAULT 'PENDING'
    CHECK (status IN ('PENDING','CONFIRMED','CANCELLED','COMPLETED')),
  CHECK (end_date >= start_date),
  FOREIGN KEY (customer_id)      REFERENCES CUSTOMER(customer_id),
  FOREIGN KEY (vehicle_id)       REFERENCES VEHICLE(vehicle_id),
  FOREIGN KEY (pickup_branch_id) REFERENCES BRANCH(branch_id)
);

CREATE TABLE RENTAL (
  rental_id       VARCHAR(10) PRIMARY KEY,
  reservation_id  VARCHAR(10) NOT NULL UNIQUE,
  pickup_datetime DATETIME NOT NULL,
  odometer_out    INT NOT NULL,
```

```
  fuel_out        VARCHAR(10),
  deposit         DECIMAL(8,2) NOT NULL CHECK (deposit >= 0),
  FOREIGN KEY (reservation_id) REFERENCES RESERVATION(reservation_id)
);

CREATE TABLE RETURN_RECORD (
  return_id        VARCHAR(10) PRIMARY KEY,
  rental_id        VARCHAR(10) NOT NULL UNIQUE,
  return_branch_id VARCHAR(10) NOT NULL,
  return_datetime  DATETIME NOT NULL,
  odometer_in      INT NOT NULL,
  damage_notes     VARCHAR(100),
  late_fee         DECIMAL(8,2) DEFAULT 0 CHECK (late_fee >= 0),
  FOREIGN KEY (rental_id)        REFERENCES RENTAL(rental_id),
  FOREIGN KEY (return_branch_id) REFERENCES BRANCH(branch_id)
);

CREATE TABLE PAYMENT (
  payment_id VARCHAR(10) PRIMARY KEY,
  rental_id  VARCHAR(10) NOT NULL,
  amount     DECIMAL(8,2) NOT NULL CHECK (amount > 0),
  mode       VARCHAR(20) NOT NULL,
  status     VARCHAR(15) NOT NULL,
  FOREIGN KEY (rental_id) REFERENCES RENTAL(rental_id)
);

-- Indexes for fast search
CREATE INDEX idx_vehicle_branch ON VEHICLE(branch_id);
CREATE INDEX idx_resv_vehicle_dates ON RESERVATION(vehicle_id, start_date, end_date);
CREATE INDEX idx_resv_customer ON RESERVATION(customer_id);
```

After execution, `SHOW TABLES` lists the seven tables of the system (Fig. 2).

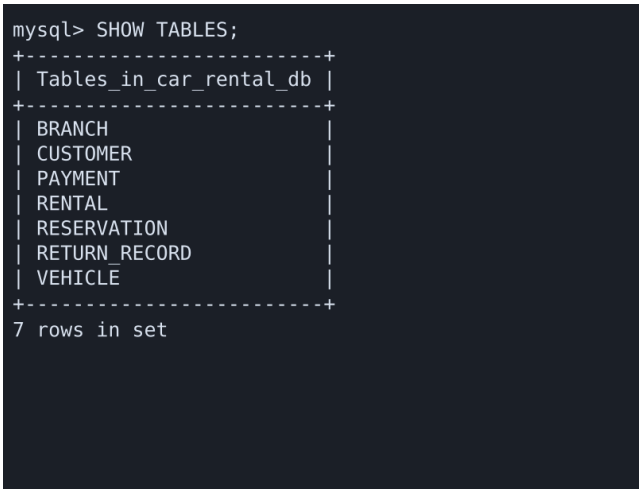


Fig. 2. MySQL-style output of SHOW TABLES after running the DDL script.

2) DML Commands and Sample Data

Data Manipulation Language commands insert, retrieve, update and delete records. The following statements load the sample data used throughout the report.

```
INSERT INTO CUSTOMER (customer_id,name,phone,license_no,city) VALUES
  ('c01','Rahul Verma','9876543210','TS0920210045','Hyderabad'),
  ('c02','Ananya Roy','9123456780','KA0120190872','Bengaluru'),
  ('c03','Karthik Nair','9988776655','TS0720180331','Hyderabad');

INSERT INTO BRANCH VALUES
  ('br01','Hyderabad Airport Hub','Hyderabad','Telangana','ACTIVE'),
  ('br02','Koramangala Branch','Bengaluru','Karnataka','ACTIVE');

INSERT INTO VEHICLE (vehicle_id,reg_no,make_model,category,daily_rate,branch_id)
VALUES
  ('v101','TS09EA4521','Hyundai Creta','SUV',2500.00,'br01'),
```

```
  ('v102','KA01MJ7788','Maruti Swift','Hatchback',1200.00,'br02'),
  ('v103','TS07FK1190','Toyota Innova','MPV',3000.00,'br01');

INSERT INTO RESERVATION VALUES
  ('r701','c01','v101','br01','2026-10-18','2026-10-21','CONFIRMED'),
  ('r702','c02','v102','br02','2026-10-19','2026-10-20','PENDING');

INSERT INTO RENTAL VALUES ('rt901','r701','2026-10-18
10:00:00',18240,'Full',5000.00);

INSERT INTO RETURN_RECORD
(return_id,rental_id,return_branch_id,return_datetime,odometer_in,damage_notes)
  VALUES ('ret01','rt901','br01','2026-10-21 12:30:00',18795,'None');   -- late_fee
set by trigger

INSERT INTO PAYMENT VALUES ('pay01','rt901',8000.00,'UPI','SUCCESS');

-- Other DML examples
UPDATE CUSTOMER SET city = 'Secunderabad' WHERE customer_id = 'c03';
UPDATE RESERVATION SET status = 'CANCELLED' WHERE reservation_id = 'r702';
DELETE FROM PAYMENT WHERE payment_id = 'pay_test';
```

The contents of the tables after loading the data and executing the return (including the values set automatically by the triggers) are shown in Table XX to Table XXVI.

TABLE XX. CUSTOMER TABLE (SAMPLE DATA)

customer_id	name	phone	email	license_no	city
c01	Rahul Verma	9876543210	NULL	TS0920210045	Hyderabad
c02	Ananya Roy	9123456780	NULL	KA0120190872	Bengaluru
c03	Karthik Nair	9988776655	NULL	TS0720180331	Hyderabad

TABLE XXI. BRANCH TABLE (SAMPLE DATA)

branch_id	branch_name	city	state	status
br01	Hyderabad Airport Hub	Hyderabad	Telangana	ACTIVE
br02	Koramangala Branch	Bengaluru	Karnataka	ACTIVE

TABLE XXII. VEHICLE TABLE (SAMPLE DATA, AFTER RETURN)

vehicle_id	reg_no	make_model	category	daily_rate	branch_id	status
v101	TS09EA4521	Hyundai Creta	SUV	2,500.00	br01	AVAILABLE
v102	KA01MJ7788	Maruti Swift	Hatchback	1,200.00	br02	AVAILABLE
v103	TS07FK1190	Toyota Innova	MPV	3,000.00	br01	AVAILABLE

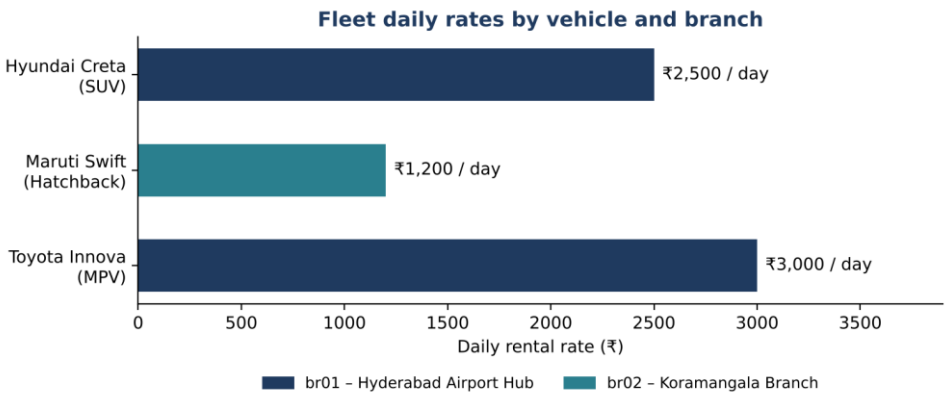


Fig. 3. Daily rental rate of each vehicle in the fleet, grouped by home branch.

TABLE XXIII. RESERVATION TABLE (SAMPLE DATA, AFTER RETURN)

reservation_id	customer_id	vehicle_id	pickup_branch_id	start_date	end_date	status
r701	c01	v101	br01	2026-10-18	2026-10-21	COMPLETED
r702	c02	v102	br02	2026-10-19	2026-10-20	PENDING

TABLE XXIV. RENTAL TABLE (SAMPLE DATA)

rental_id	reservation_id	pickup_datetime	odometer_out	fuel_out	deposit
rt901	r701	2026-10-18 10:00	18240	Full	5,000.00

TABLE XXV. RETURN_RECORD TABLE (SAMPLE DATA)

return_id	rental_id	return_branch_id	return_datetime	odometer_in	damage_notes	late_fee
ret01	rt901	br01	2026-10-21 12:30	18795	None	500.00

TABLE XXVI. PAYMENT TABLE (SAMPLE DATA)

payment_id	rental_id	amount	mode	status
pay01	rt901	8,000.00	UPI	SUCCESS

Note that for rental `rt901` the trigger has automatically set `late_fee` to 500.00 because the car was returned at 12:30 on 21 October, after the due time of 10:00 on the end date. The reservation `r701` is now COMPLETED and the vehicle `v101` is AVAILABLE again.

3) Sample Queries

The following queries demonstrate how the database supports key operations. Each query was executed on the sample data and the output is shown below it.

Q1. Find available vehicles at a branch for given dates

```
SELECT v.vehicle_id, v.make_model, v.category, v.daily_rate
FROM VEHICLE v
WHERE v.branch_id = 'br01' AND v.status = 'AVAILABLE'
  AND v.vehicle_id NOT IN (
      SELECT vehicle_id FROM RESERVATION
      WHERE status <> 'CANCELLED'
        AND start_date <= '2026-10-21' AND end_date >= '2026-10-18');
```

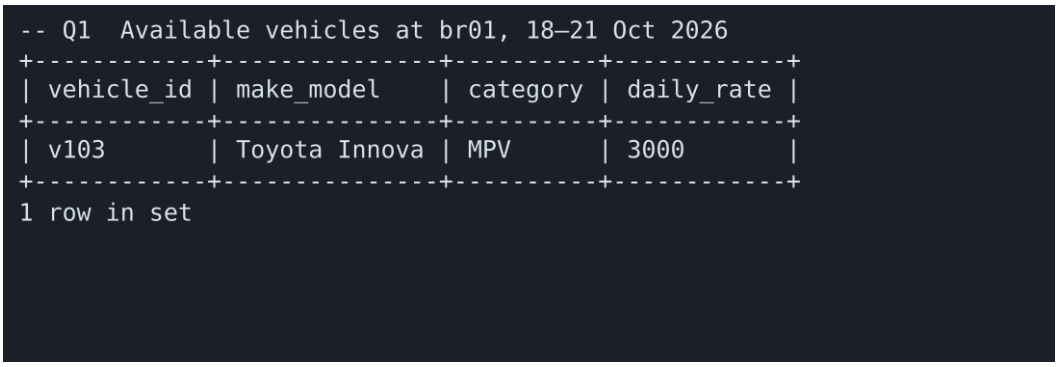


Fig. 4. Output of Q1.

Only the Toyota Innova (`v103`) is free at the Hyderabad Airport Hub for 18–21 October; the Creta `v101` is excluded because reservation `r701` overlaps the requested dates. The condition `start_date <= :end AND end_date >= :start` is the standard test for overlapping date ranges.

Q2. Return summary with late fee

```
SELECT rt.return_id, c.name, v.reg_no, rt.return_datetime, rt.late_fee
```

```
FROM RETURN_RECORD rt
JOIN RENTAL r      USING (rental_id)
JOIN RESERVATION rs USING (reservation_id)
JOIN CUSTOMER c    USING (customer_id)
JOIN VEHICLE v     USING (vehicle_id)
WHERE r.rental_id = 'rt901';
```

```
-- Q2  Return summary for rental rt901
+-----------+------------+------------+-----------------+----------+
| return_id | name       | reg_no     | return_datetime | late_fee |
+-----------+------------+------------+-----------------+----------+
| ret01     | Rahul Verma | TS09EA4521 | 2026-10-21 12:30 | 500      |
+-----------+------------+------------+-----------------+----------+
1 row in set
```

Fig. 5. Output of Q2.

Q3. Total revenue per branch

```
SELECT b.branch_name, SUM(p.amount) AS revenue
FROM PAYMENT p
JOIN RENTAL r       USING (rental_id)
JOIN RESERVATION rs USING (reservation_id)
JOIN BRANCH b       ON rs.pickup_branch_id = b.branch_id
WHERE p.status = 'SUCCESS'
GROUP BY b.branch_name;
```

```
-- Q3  Revenue per branch
+----------------------+---------+
| branch_name          | revenue |
+----------------------+---------+
| Hyderabad Airport Hub | 8000    |
+----------------------+---------+
1 row in set
```

Fig. 6. Output of Q3.

Q4. Number of reservations per customer

```
SELECT c.name, COUNT(rs.reservation_id) AS total_reservations
FROM CUSTOMER c
LEFT JOIN RESERVATION rs ON rs.customer_id = c.customer_id
GROUP BY c.customer_id, c.name
ORDER BY total_reservations DESC, c.name;
```

```
-- Q4  Reservations per customer
+-------------+--------------------+
| name        | total_reservations |
+-------------+--------------------+
| Ananya Roy  | 1                  |
| Rahul Verma | 1                  |
| Karthik Nair | 0                 |
+-------------+--------------------+
3 rows in set
```

Fig. 7. Output of Q4.

The LEFT JOIN keeps customers with no reservations (Karthik Nair) in the result with a count of zero.

Q5. Fleet size and average rate by branch and category

```
SELECT b.branch_name, v.category, COUNT(*) AS vehicles,
       ROUND(AVG(v.daily_rate),2) AS avg_daily_rate
FROM VEHICLE v JOIN BRANCH b USING (branch_id)
GROUP BY b.branch_name, v.category
ORDER BY b.branch_name;
```

```
-- Q5  Fleet by branch and category
+----------------------+-----------+----------+----------------+
| branch_name          | category  | vehicles | avg_daily_rate |
+----------------------+-----------+----------+----------------+
| Hyderabad Airport Hub | MPV       | 1        | 3000.00        |
| Hyderabad Airport Hub | SUV       | 1        | 2500.00        |
| Koramangala Branch    | Hatchback | 1        | 1200.00        |
+----------------------+-----------+----------+----------------+
3 rows in set
```

Fig. 8. Output of Q5.

Q6. Pending reservations awaiting confirmation

```
SELECT rs.reservation_id, c.name, v.make_model, rs.start_date, rs.status
FROM RESERVATION rs
JOIN CUSTOMER c USING (customer_id)
JOIN VEHICLE v  USING (vehicle_id)
WHERE rs.status = 'PENDING';
```

```
-- Q6  Pending reservations
+---------------+------------+--------------+------------+---------+
| reservation_id | name       | make_model   | start_date | status  |
+---------------+------------+--------------+------------+---------+
| r702          | Ananya Roy | Maruti Swift | 2026-10-19 | PENDING |
+---------------+------------+--------------+------------+---------+
1 row in set
```

Fig. 9. Output of Q6.

Q7. Final bill for a rental (rental charge + late fee)

```
SELECT r.rental_id,
       (rt.odometer_in - r.odometer_out) AS km_driven,
       DATEDIFF(rs.end_date, rs.start_date) * v.daily_rate AS base_charge,
       rt.late_fee,
       DATEDIFF(rs.end_date, rs.start_date) * v.daily_rate + rt.late_fee AS
total_bill
FROM RENTAL r
JOIN RETURN_RECORD rt USING (rental_id)
JOIN RESERVATION rs   USING (reservation_id)
JOIN VEHICLE v        USING (vehicle_id);
```

```
-- Q7  Bill for rental rt901
+-----------+-----------+-------------+----------+------------+
| rental_id | km_driven | base_charge | late_fee | total_bill |
+-----------+-----------+-------------+----------+------------+
| rt901     | 555       | 7500        | 500      | 8000       |
+-----------+-----------+-------------+----------+------------+
1 row in set
```

Fig. 10. Output of Q7.

For `rt901` the customer drove 18795 − 18240 = **555 km**. The base charge is 3 days × ₹2,500 = ₹7,500, the late fee is ₹500 and the total bill is **₹8,000**, which matches the successful UPI payment `pay01`. The ₹5,000 security deposit is held separately and released after the damage inspection (no damage was recorded). Fig. 11 visualizes the composition of the bill.

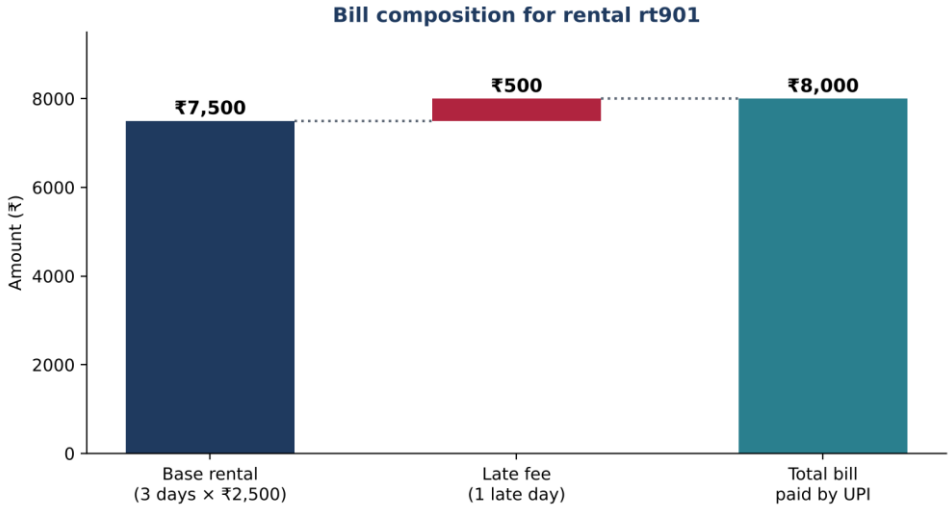


Fig. 11. Composition of the final bill for rental rt901 (base charge, late fee and total).

C. Database Connectivity

The application layer reaches the database through a connector library. In the proposed Python backend, a small module creates one connection per request using `mysql-connector-python`, and every business operation (search, reserve, pickup, return, pay) calls a stored SQL statement or transaction from this module. The snippet below is illustrative of this design.

```
import mysql.connector

def get_connection():
    return mysql.connector.connect(
        host="localhost", user="rental_app", password="********",
        database="car_rental_db")

def available_vehicles(branch_id, start, end):
    con = get_connection(); cur = con.cursor(dictionary=True)
    cur.execute(AVAILABILITY_SQL, (branch_id, end, start))   # Q1 with parameters
    rows = cur.fetchall(); con.close()
    return rows
```

Using parameterized statements (placeholders instead of string concatenation) protects the database against SQL injection, and a dedicated low-privilege database user (`rental_app`) is used instead of the root account.

D. Triggers, Transactions and Concurrency

Beyond simple constraints, the rental business has rules that depend on several tables. These are enforced inside the database so that every application obtains the same behaviour.

1) Triggers

Four triggers automate the rental workflow (Table XXVII).

TABLE XXVII. TRIGGERS USED IN THE DATABASE

Trigger	Event	Action
trg_no_overlap	BEFORE INSERT on RESERVATION	Rejects a booking if the same vehicle already has a non-cancelled reservation on overlapping dates
trg_rental_out	AFTER INSERT on RENTAL	Sets the vehicle status to RENTED
trg_return_fee	BEFORE INSERT on RETURN_RECORD	Calculates late_fee: ₹500 per late day or part of a day after 10:00 on the reservation end date
trg_return_close	AFTER INSERT on RETURN_RECORD	Sets the vehicle to AVAILABLE and the reservation to COMPLETED

Business rule used for the sample data: a car must be returned by 10:00 on the end date of the reservation; each late day or part of a day is charged ₹500. With this rule, a return at 12:30 on the end date incurs one late day (₹500), as in rental `rt901`.

```
DELIMITER //
CREATE TRIGGER trg_no_overlap BEFORE INSERT ON RESERVATION
FOR EACH ROW
BEGIN
  IF NEW.status <> 'CANCELLED' AND EXISTS (
       SELECT 1 FROM RESERVATION r
       WHERE r.vehicle_id = NEW.vehicle_id AND r.status <> 'CANCELLED'
         AND r.start_date <= NEW.end_date AND r.end_date >= NEW.start_date) THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Vehicle already reserved for overlapping dates';
  END IF;
END//

CREATE TRIGGER trg_rental_out AFTER INSERT ON RENTAL
FOR EACH ROW
BEGIN
  UPDATE VEHICLE SET status = 'RENTED'
  WHERE vehicle_id = (SELECT vehicle_id FROM RESERVATION
                      WHERE reservation_id = NEW.reservation_id);
END//

CREATE TRIGGER trg_return_fee BEFORE INSERT ON RETURN_RECORD
FOR EACH ROW
BEGIN
  DECLARE v_due DATETIME;
  SELECT TIMESTAMP(rs.end_date, '10:00:00') INTO v_due
  FROM RENTAL r JOIN RESERVATION rs ON rs.reservation_id = r.reservation_id
  WHERE r.rental_id = NEW.rental_id;
  IF NEW.return_datetime > v_due THEN
    SET NEW.late_fee = 500 * CEIL(TIMESTAMPDIFF(MINUTE, v_due, NEW.return_datetime) /
1440);
  ELSE
    SET NEW.late_fee = 0;
  END IF;
END//

CREATE TRIGGER trg_return_close AFTER INSERT ON RETURN_RECORD
FOR EACH ROW
BEGIN
  UPDATE VEHICLE SET status = 'AVAILABLE'
  WHERE vehicle_id = (SELECT rs.vehicle_id FROM RENTAL r
       JOIN RESERVATION rs ON rs.reservation_id = r.reservation_id
       WHERE r.rental_id = NEW.rental_id);
```

```
  UPDATE RESERVATION SET status = 'COMPLETED'
  WHERE reservation_id = (SELECT reservation_id FROM RENTAL WHERE rental_id =
NEW.rental_id);
END//
DELIMITER ;
```

The status changes caused by the application and the triggers follow the state diagrams in Fig. 12.

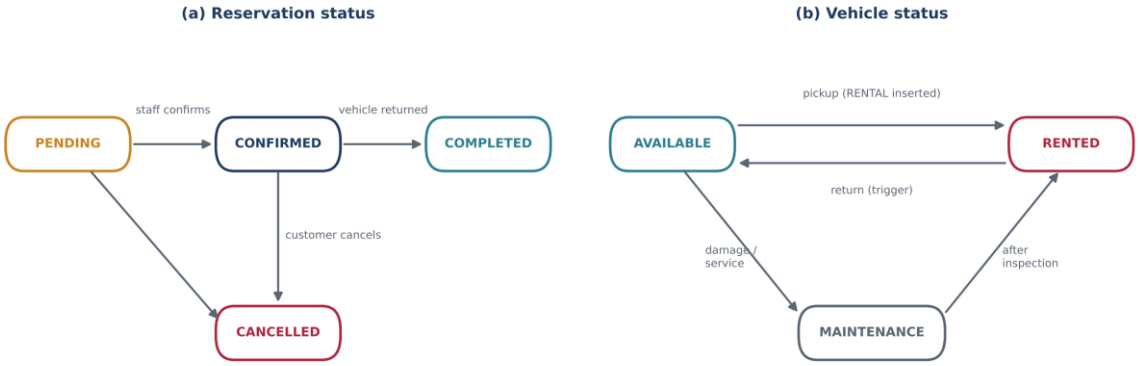


Fig. 12. State diagrams for (a) reservation status and (b) vehicle status.

2) Transaction Management (ACID)

Reservation, vehicle blocking and deposit must succeed or fail together. The booking is therefore executed as a single transaction (Fig. 13): the vehicle row is locked with `SELECT … FOR UPDATE`, the overlap rule is checked, the reservation and payment records are inserted and the transaction is committed only if the payment succeeds; otherwise everything is rolled back.

```
START TRANSACTION;
  SELECT status FROM VEHICLE WHERE vehicle_id = 'v103' FOR UPDATE;   -- lock the car
  INSERT INTO RESERVATION VALUES ('r703','c03','v103','br01','2026-10-25','2026-10-
27','CONFIRMED');
  INSERT INTO RENTAL VALUES ('rt902','r703',NOW(),9000,'Full',5000.00);
  INSERT INTO PAYMENT VALUES ('pay02','rt902',5000.00,'CARD','SUCCESS');
COMMIT;   -- or ROLLBACK; if any step fails
```

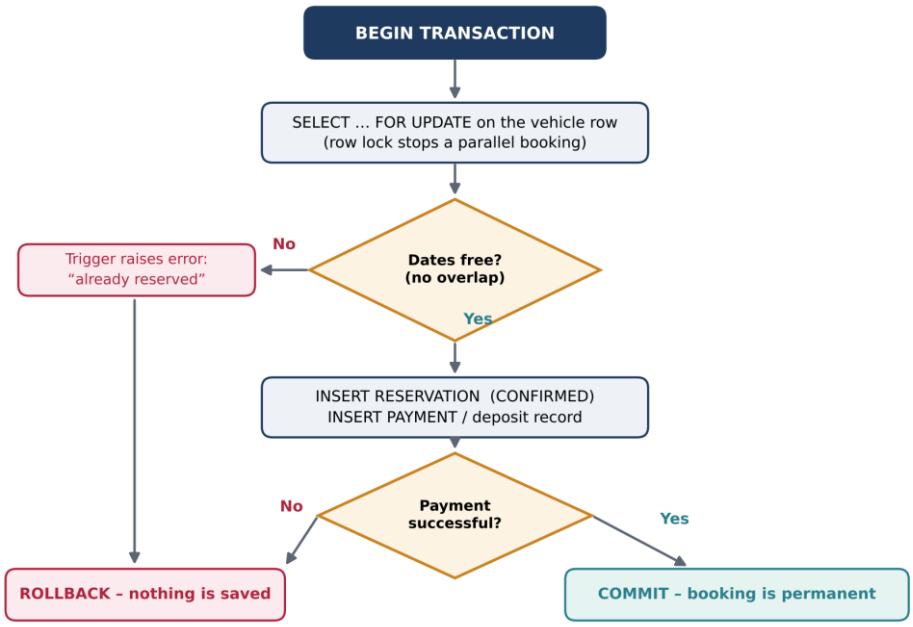


Fig. 13. Booking transaction flow with row locking, trigger validation and rollback.

TABLE XXVIII. HOW THE ACID PROPERTIES ARE ACHIEVED

Property	Meaning	Mechanism in this system
Atomicity	All steps of a booking or return succeed, or none do	START TRANSACTION … COMMIT / ROLLBACK
Consistency	Database moves from one valid state to another	Constraints and triggers validate every change
Isolation	Concurrent users do not see partial work	Row locks (FOR UPDATE) prevent two users reserving the same car
Durability	Committed data survives failures	InnoDB redo log and regular backups

E. Validation and Error Handling

Invalid operations are rejected by the database itself, so that incorrect data can never be stored regardless of which application sends it. For example, deleting a customer who is referenced by a reservation violates a foreign-key constraint (MySQL error 1451); inserting a vehicle with a zero daily rate violates a CHECK constraint (error 3819); and an overlapping reservation is rejected by the trigger with SQLSTATE 45000. The application catches these errors and shows a meaningful message instead of terminating. This behaviour also demonstrates that referential and domain integrity are correctly enforced. The complete set of validation tests is given in Section VII.

F. System Architecture and Data Flow

The system follows a three-layer architecture (Fig. 14). Users (customers, branch staff and administrators) interact with the presentation layer, which sends requests to the application layer. The application layer contains the reservation engine, availability checker, fare calculator, return processor and billing module, and talks to the RDBMS, which stores the seven tables and enforces constraints, indexes, triggers and transactions.

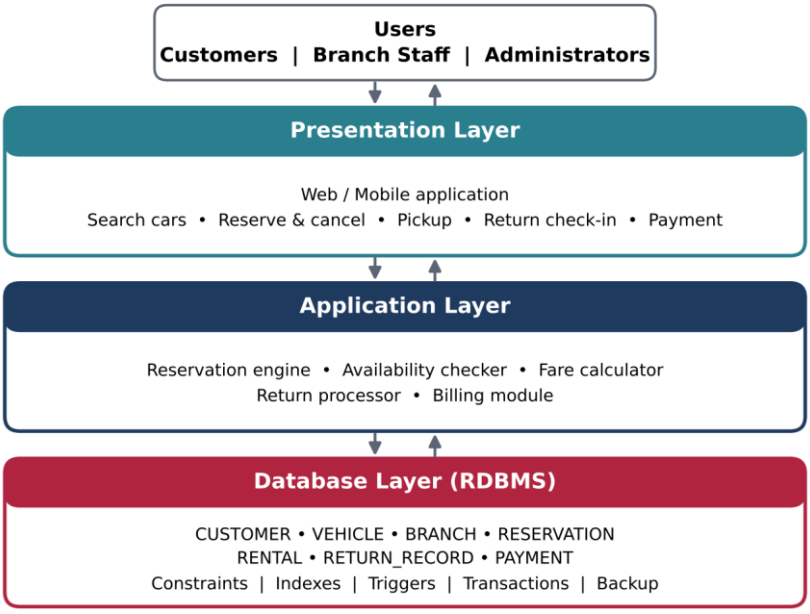


Fig. 14. Three-layer system architecture of the Car Rental Reservation and Vehicle Return Management System.

The end-to-end data flow of a rental is shown in Fig. 15. A customer searches for a vehicle (availability query), the reservation is created and confirmed, the rental agreement is issued at pickup, the return is checked in, the late-fee trigger runs, the bill is settled by a payment and finally the vehicle and the reservation are closed.

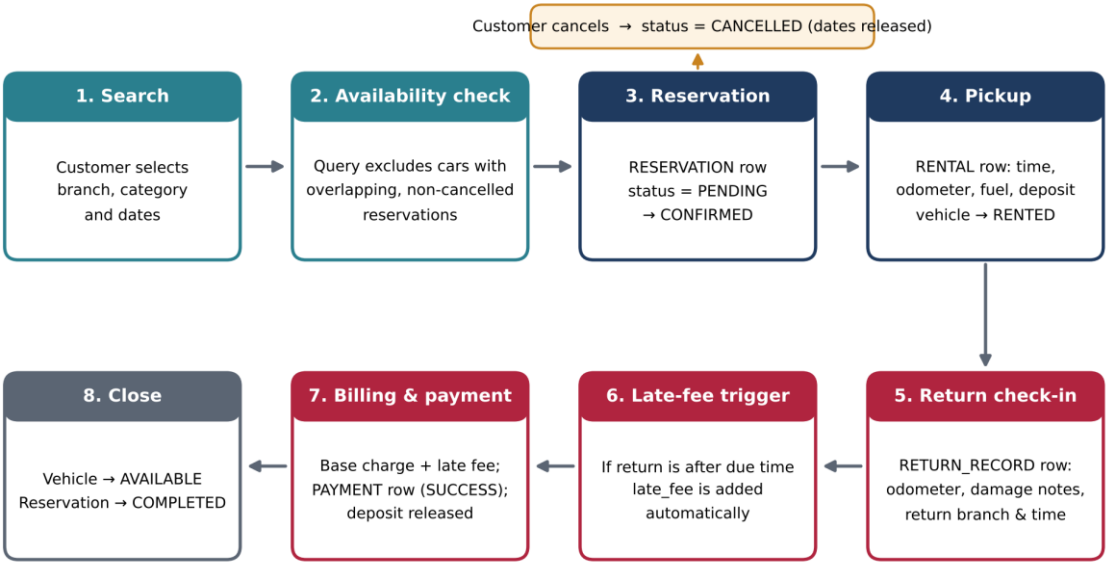


Fig. 15. Lifecycle and data flow of a rental, from search to closing of the reservation.

G. Security, Indexing and Backup

- **Role-based access:** separate database users for customer-facing services, branch staff and administrators, each with only the privileges listed in Table III (GRANT SELECT / INSERT / UPDATE on selected tables).
- **Encrypted sensitive data:** payment details and licence numbers are encrypted at rest and in transit (TLS).
- **Indexes:** indexes on `VEHICLE(branch_id)`, `RESERVATION(vehicle_id, start_date, end_date)` and `RESERVATION(customer_id)` speed up availability search and customer history queries.
- **Backup and recovery:** regular full backups with `mysqldump` plus binary logs for point-in-time recovery.
- **SQL-injection protection:** parameterized queries and a least-privilege application account.

VII. Testing

The database was tested using valid and invalid inputs, including constraint-related and trigger-related cases. The tests were run on the sample data of Section VI; each test was executed inside a transaction that was rolled back afterwards so that the data remained unchanged. The tests were executed on an SQLite-compatible replica of the schema; the error numbers shown are those returned by MySQL 8.0 for the same violations. Table XXIX lists the constraint and integrity tests and Table XXX the trigger and transaction tests.

TABLE XXIX. CONSTRAINT AND INTEGRITY TEST CASES

#	Test case	Input / action	Expected result	System response	Result
1	Add customer	Insert customer with unique phone and licence	Record accepted	—	Pass
2	Duplicate phone	Insert customer with an existing phone number	Rejected	Error 1062 (duplicate entry)	Pass
3	Duplicate licence	Insert customer with an existing licence number	Rejected	Error 1062 (duplicate entry)	Pass
4	Invalid daily rate	Insert vehicle with daily_rate = 0	Rejected	Error 3819 (CHECK violated)	Pass
5	Invalid date range	Reservation with end_date before start_date	Rejected	Error 3819 (CHECK violated)	Pass
6	Overlapping booking	Book v101 for 2026-10-20 to 2026-10-22 (overlaps r701)	Rejected	SQLSTATE 45000 (trigger)	Pass
7	Non-overlapping booking	Book v101 for 2026-10-22 to 2026-10-24	Record accepted	—	Pass
8	Unknown customer	Reservation for non-existent customer c99	Rejected	Error 1452 (FK violated)	Pass
9	Delete referenced customer	Delete customer c01 who has a reservation	Rejected	Error 1451 (FK violated)	Pass
10	Invalid status	Set reservation status to an undefined value	Rejected	Error 3819 (CHECK violated)	Pass
11	Negative payment	Insert payment with amount -100	Rejected	Error 3819 (CHECK violated)	Pass

TABLE XXX. TRIGGER AND TRANSACTION TEST CASES

#	Test case	Input / action	Expected result	Result
12	Vehicle blocked at pickup	Insert RENTAL for reservation r703 (v103)	v103 status = RENTED	Pass
13	On-time return	Return 24 Oct 09:30 (due 24 Oct 10:00)	late_fee = 0.00	Pass
14	Late return, multi-day	Return 25 Oct 15:00 (due 24 Oct 10:00)	late_fee = 1000.00 (2 late days)	Pass
15	Vehicle released at return	After the return in test 13	v103 status = AVAILABLE	Pass
16	Late fee on sample data	Return of rt901 at 12:30 on 21 Oct	late_fee = 500.00 and r701 = COMPLETED	Pass
17	Transaction rollback	Reservation inserted, then payment fails (invalid rental)	Reservation r704 not stored	Pass

All 17 test cases produced the expected result. The tests confirm that the database prevents duplicate customers and vehicles, invalid rates and dates, orphan records, deletion of referenced parents and double booking, and that the triggers compute late fees and statuses correctly.

VIII. Conclusion and Future Work

The Car Rental Reservation and Vehicle Return Management System demonstrates the practical application of database management concepts to a real business problem. A complete database design was produced for the reservation and return workflow: seven core entities were identified, an ER diagram and a 3NF relational schema were designed, and data integrity was enforced with primary, foreign, unique, NOT NULL and CHECK constraints. Double booking of vehicles is prevented by an overlap trigger together with row locking inside ACID transactions, while triggers calculate late fees and keep vehicle and reservation statuses consistent. SQL queries for availability search, return summaries, revenue per branch and billing show that the design supports the day-to-day operations and management reporting of a rental company.

Key achievements

- Identified seven core entities and their relationships.
- Designed the ER diagram and a 3NF schema.
- Enforced integrity with constraints and triggers.
- Prevented vehicle double booking.
- Automated late-fee billing and status updates.
- Verified the design with sample data and 17 test cases.

System benefits: centralized data storage, efficient queries and reporting, enforced data consistency and a design that is scalable for future enhancements.

Future work

- User interface development: web and mobile application for customers and staff
- Query optimization and additional indexing for large fleets
- Dynamic pricing based on demand, season and category
- GPS tracking of vehicles and maintenance alerts
- Online payment-gateway integration and e-mail / SMS confirmations
- Deployment on a cloud server with automated backups and monitoring
- Admin dashboard with utilization, revenue and damage analytics

Contribution of Team Members

The project was carried out as a team of four students. The individual contributions are recorded below.

TABLE XXXI. CONTRIBUTION OF TEAM MEMBERS

Team member	Roll No.	Contribution
Vaibhav	4018	[Describe contribution – e.g. requirement analysis, ER design]
V Charan	2296	[Describe contribution – e.g. relational schema, normalization]
A Karthikeya	3001	[Describe contribution – e.g. SQL scripts, triggers, queries]
Y. Charan	2313	[Describe contribution – e.g. testing, documentation, presentation]

Appendix A: Source Code Repository

The SQL scripts of the project are organized as follows. Repository link: **[add GitHub URL here]**.

TABLE XXXII. CONTENTS OF THE SOURCE CODE REPOSITORY

File	Contents
`schema.sql`	CREATE DATABASE, seven CREATE TABLE statements, constraints and indexes (Section VI.B.1)
`triggers.sql`	trg_no_overlap, trg_rental_out, trg_return_fee, trg_return_close (Section VI.D)
`sample_data.sql`	INSERT statements for the sample customers, branches, vehicles, reservations, rental, return and payment (Section VI.B.2)
`queries.sql`	Queries Q1 to Q7 (Section VI.B.3)
`tests.sql`	Constraint, trigger and rollback test cases (Section VII)
`README.md`	Instructions to create the database and run the scripts in MySQL 8.0

To recreate the database in MySQL: run `schema.sql`, then `triggers.sql`, then `sample_data.sql`, and finally `queries.sql`.

References

[1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

[2] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.

[3] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. McGraw-Hill, 2003.

[4] Oracle Corporation, *MySQL 8.0 Reference Manual* – CREATE TABLE, CHECK constraints, triggers and InnoDB locking. https://dev.mysql.com/doc/refman/8.0/en/

[5] P. P. Chen, "The Entity-Relationship Model – Toward a Unified View of Data," *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.

[6] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.